Machine Learning

Machine learning with Probability

Dong-Kyu Chae

PI of the Data Intelligence Lab @HYU
Department of Computer Science
Hanyang University

Data
Intelligence
LAB

Why We need Probability & Statistics?

- Real world data is "uncertain" and "ambiguous"

Machine Learning with Probability

$f(\mathbf{x})$

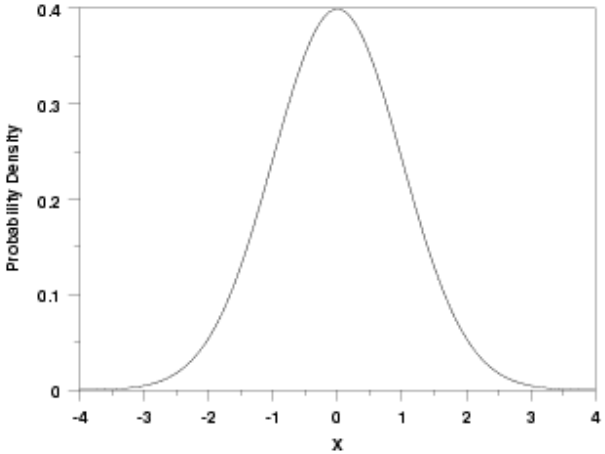
- Data -> (deterministic) model -> output

- Data -> (probabilistic) model -> output

$P(X)$

Goals (X)	Probability P(X)
0	0.18
1	0.34
2	0.35
3	0.11
4	0.02

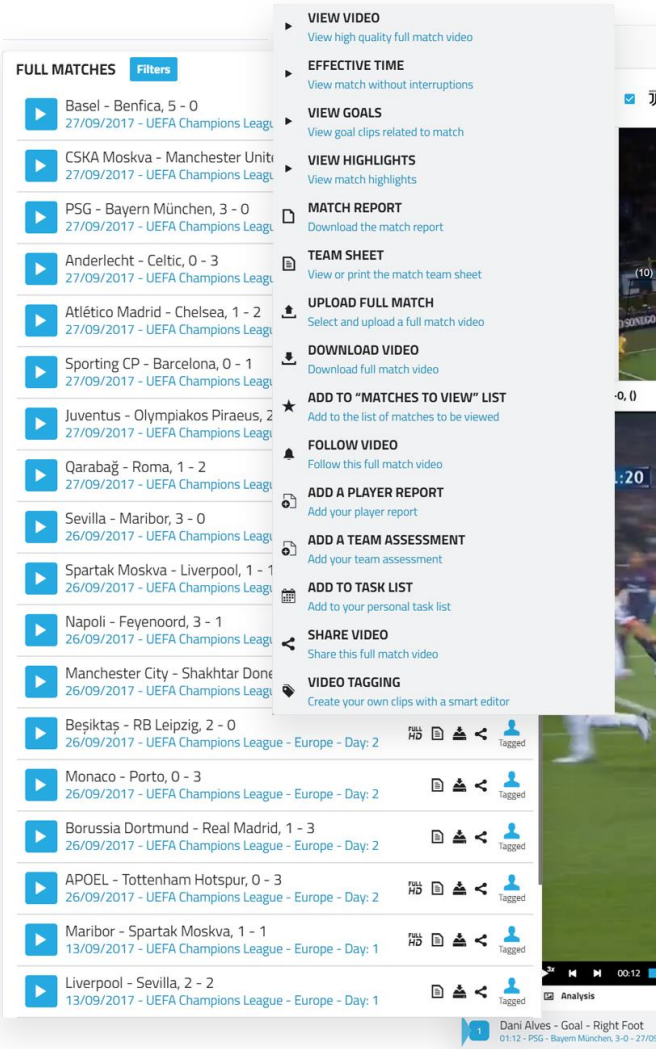
Probability mass



Probability density

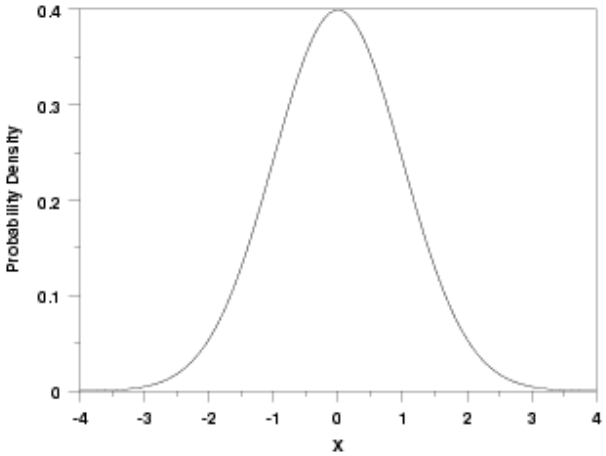
Probability Estimation

- **Data -> inferring the underlying distribution P(X)**



Goals (X)	Probability P(X)
0	0.18
1	0.34
2	0.35
3	0.11
4	0.02

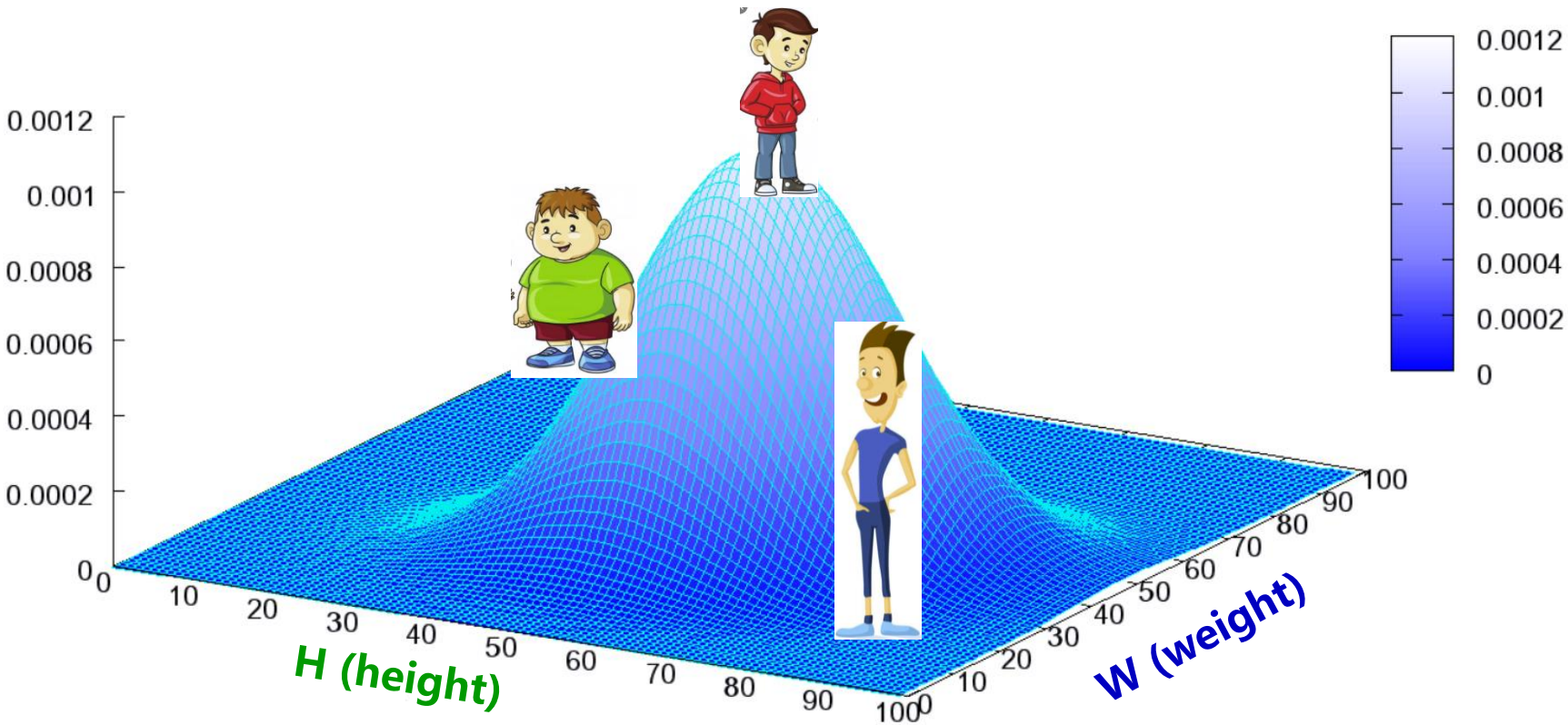
Probability mass



Probability density

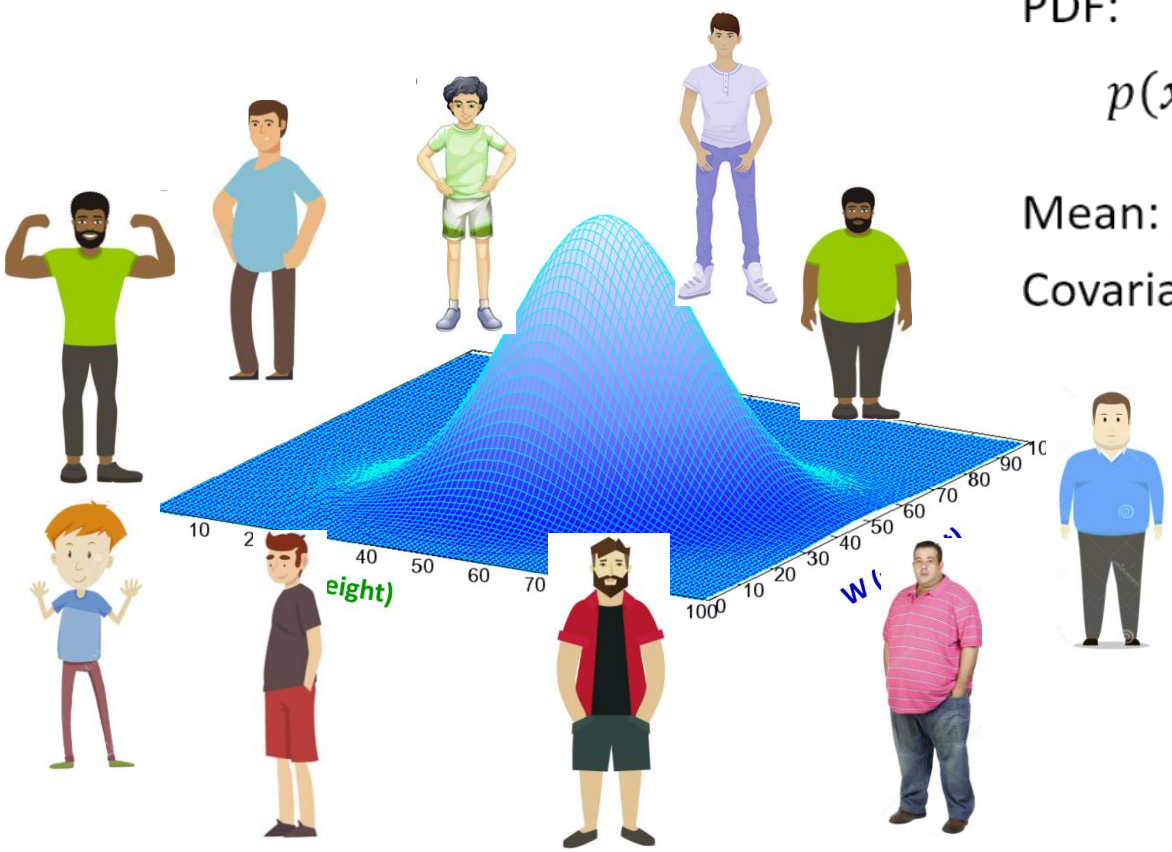
Probability Distribution

- **What is probability distribution of data?**
  - A probability distribution, P(X), is the function that maps the input X to the probabilities
    - X can be a single variable or a set of multiple variables
    - Example: X is data of men, composed of two variables: **H (height)** and **W (weight)**



Probability Distribution

- **How to learn the distribution?**
  - One way is to fit your chosen **probability density function** (e.g., Gaussian distribution) by using your training data



PDF:

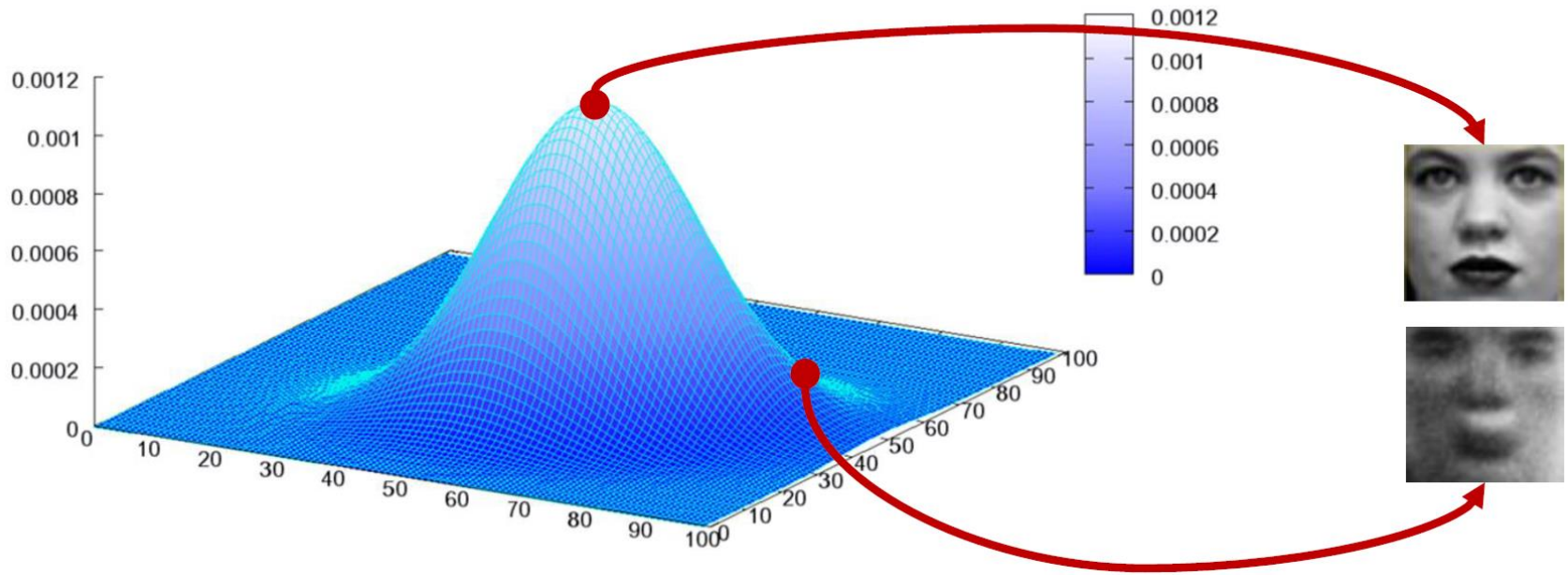
$$p(\boldsymbol{x}) = \frac{1}{\left|\sqrt{2\pi\boldsymbol{\Sigma}}\right|} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$$

Mean: $\boldsymbol{\mu}$ (vector 2x1)

Covariance: $\boldsymbol{\Sigma}$ (matrix 2x2)

Probability Distribution

- **Extending to high-dimensional data, such as images**



- We cannot visualize, but the distribution exists
- Extremely high-dimensional data, so it is difficult to explicitly estimate its distribution

What Can We Do with Probability?

- **Classification with P(X)**
  - Let X = [$\mathbf{x}$, y], where x is input feature vector and y is a class label feature
  - Then, the classification problem becomes computing $p(y|\mathbf{x})$
  - Through the Bayes theorem,

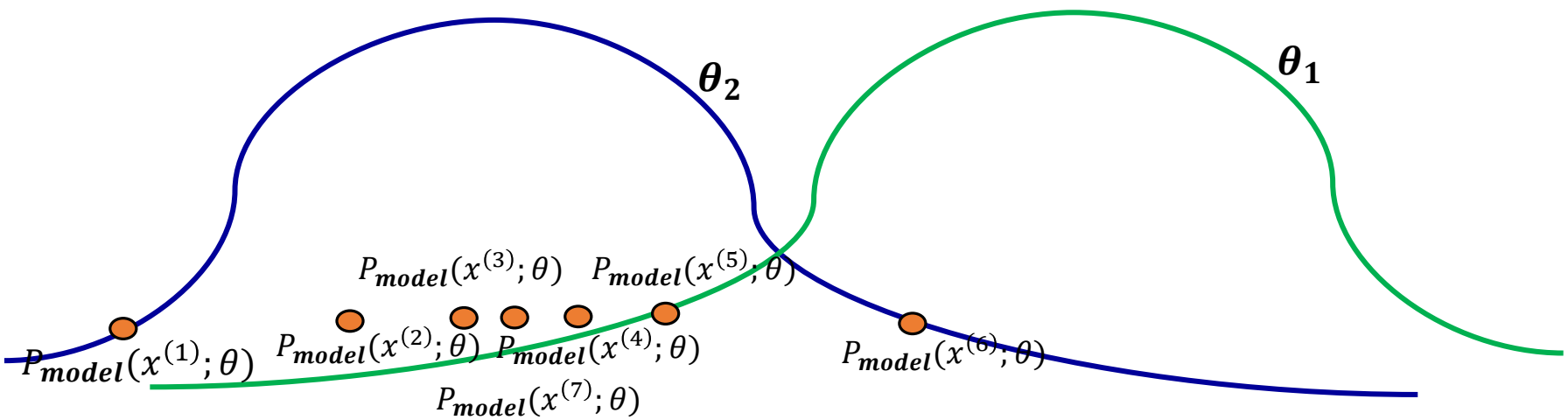
$$P(y|\mathbf{x}) = \frac{P(y,\ \mathbf{x})}{P(\mathbf{x})}$$

- **Generation with P(X)**
  - After we get P(X), we can perform random sampling based on P(X) to get a new sample
  - Or, possibly we can perform more interesting generation task
    - https://makeavideo.studio/

Maximum Likelihood Estimation (MLE)

- **MLE (maximum likelihood estimation)**
  - One of the most widely-used methods to capture the data distribution
  - It defines and optimizes a **model** that estimates probability. It is **parameterized by $\boldsymbol{\theta}$**
  - **Likelihood**: the probability that the model assigns to training data

$$\prod_{i=1}^{m} p_{\text{model}}\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right) = P_{\boldsymbol{model}}(x^{(1)}; \theta) \cdot P_{\boldsymbol{model}}(x^{(2)}; \theta) \cdots P_{\boldsymbol{model}}(x^{(m)}; \theta)$$



* Data is assumed to be generated **independently** to each other

Maximum Likelihood Estimation (MLE)

- **Likelihood: the probability that the model assigns to training data:**
  - where the training data $\boldsymbol{x}$ contains m examples
  - the result of this computation implies **the likelihood that the parameter set $\boldsymbol{\theta}$ generated the training data**

$$\prod_{i=1}^{m} p_{\text{model}}\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

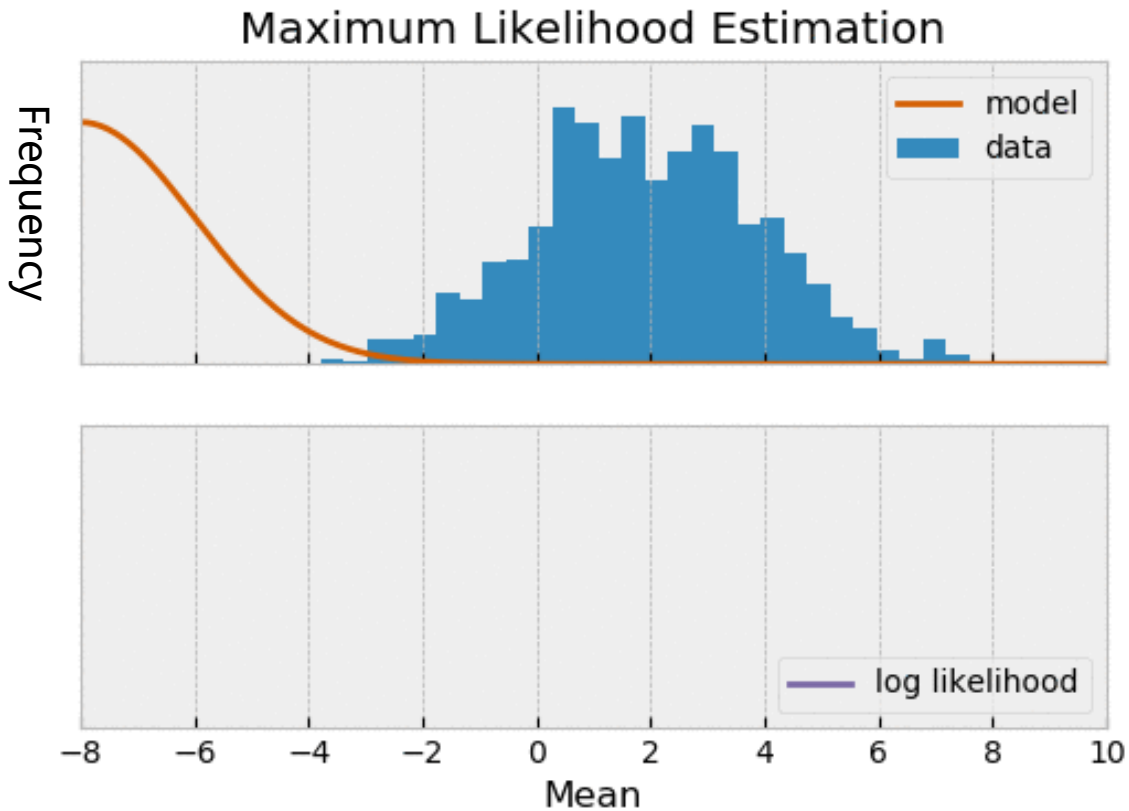
- **Maximum likelihood: to choose the parameters $\boldsymbol{\theta}$ for the model that maximize the likelihood => optimization problem!**

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p_{\text{model}}\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

- We believe that the learned model $P_{model}(x; \theta^*)$ will be similar to the true (but unknown) data distribution $P_{data}(x)$ that actually generates our training data

Maximum Likelihood Estimation (MLE)

- **MLE example: simple 1-D case**



- Model: single Gaussian distribution
- $\boldsymbol{\theta} = \{\mu,\ \sigma\}$ (mean, standard deviation)

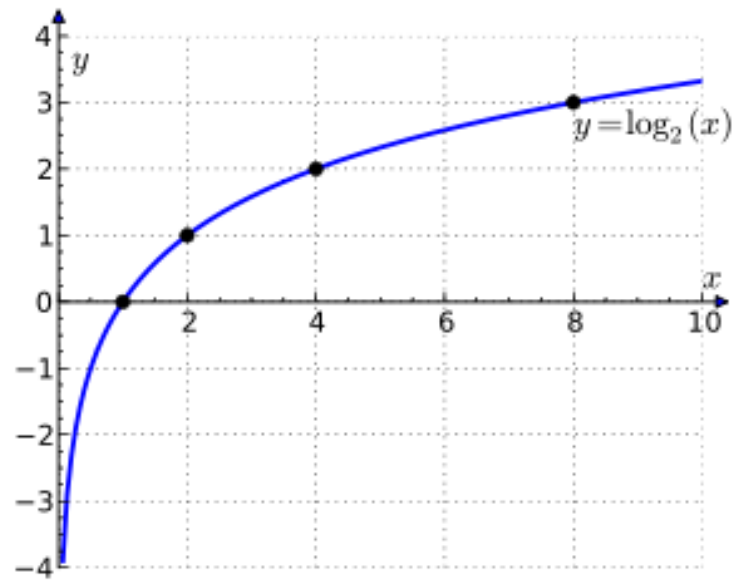
Maximum Likelihood Estimation (MLE)

- **Maximum log-likelihood**
  - The logarithm is a function that increases everywhere and does not change the location of the maximum

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} \prod_{i=1}^{m} p_{\text{model}}\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

$$= \arg\max_{\boldsymbol{\theta}} \log \prod_{i=1}^{m} p_{\text{model}}\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$

$$= \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\text{model}}\left(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right)$$



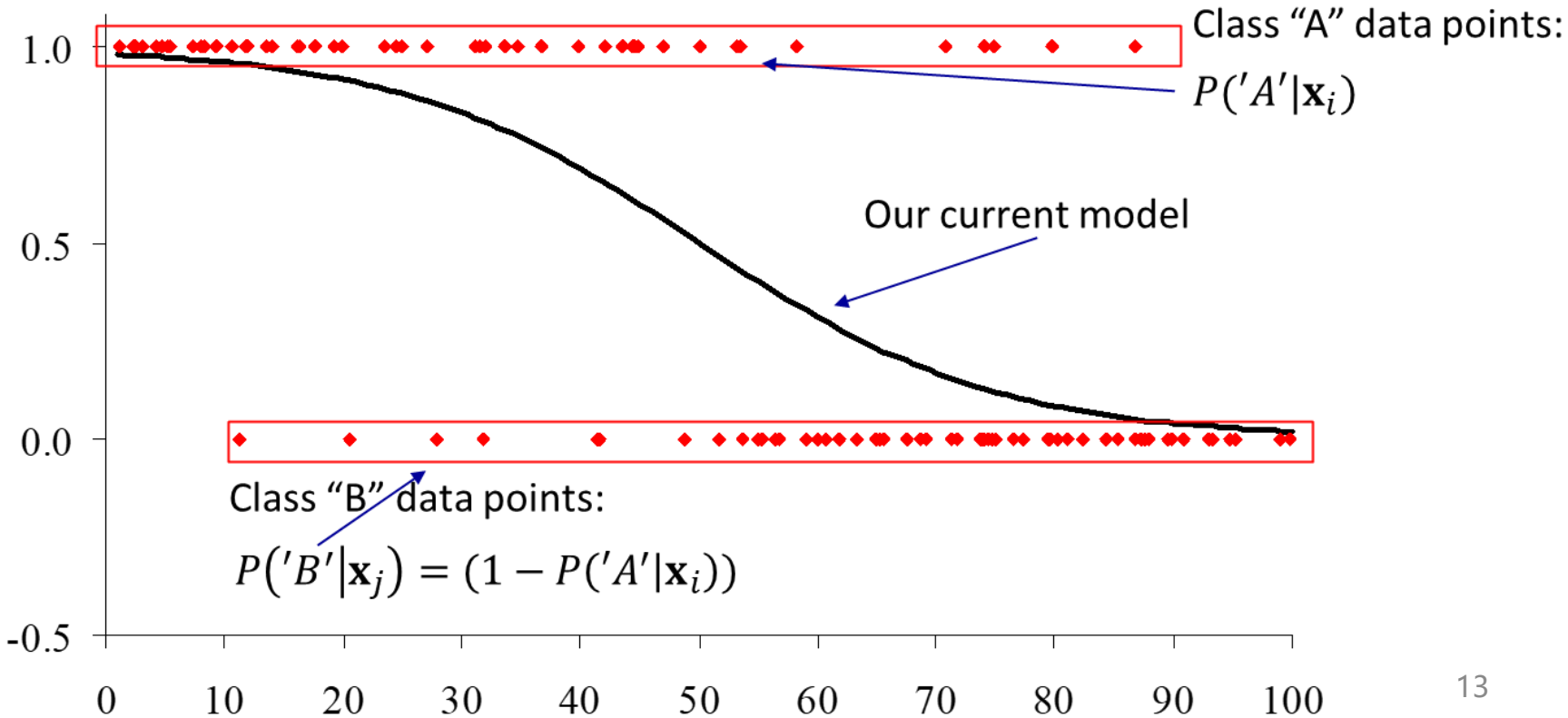
- **Maximizing log-likelihood == Minimizing negative log-likelihood**

Logistic Regression with MLE

- Estimating $\mathbf{w}$ via MLE

$\mathbf{w}$ will be omitted for simplicity

  - $\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmax}} P(y_1|\mathbf{x}_1;\mathbf{w}) \cdot P(y_2|\mathbf{x}_2;\mathbf{w}) \cdots P(y_N|\mathbf{x}_N;\mathbf{w})$
  - $P(y_1|\mathbf{x}_1) \cdot P(y_2|\mathbf{x}_2) \cdots P(y_N|\mathbf{x}_N)$ means how "likely" is our training data observed under the model distribution parameterized by $\mathbf{w}$



Logistic Regression with MLE

- MLE objective
  - MLE original: $P(y_1|\mathbf{x}_1) \cdot P(y_2|\mathbf{x}_2) \cdots P(y_N|\mathbf{x}_N) \Rightarrow \frac{1}{N}\prod_{i=1}^{N} P(y_i|\mathbf{x}_i)$
  - Decomposition
    - If y = '1' $\Rightarrow \frac{1}{N_1}\prod_{i=1}^{N_1} P(y_i|\mathbf{x}_i)$, i is iterating for data with y=1
    - If y = '0' $\Rightarrow \frac{1}{N_0}\prod_{j=1}^{N_0}(1 - P(y_j|\mathbf{x}_j))$, j is iterating for data with y=0
  - Final objective:

$$\Rightarrow \frac{1}{N}\prod_{i=1}^{N_1} P(y_i|\mathbf{x}_i) \cdot \prod_{j=1}^{N_0}(1 - P(y_j|\mathbf{x}_j))$$

$$= \frac{1}{N}\log\left(\prod_{i=1}^{N_1} P(y_i|\mathbf{x}_i) \cdot \prod_{j=1}^{N_0}(1 - P(y_j|\mathbf{x}_j))\right)$$

$$= \frac{1}{N}\left(\sum_{i=1}^{N_1} \log P(y_i|\mathbf{x}_i) + \sum_{i=1}^{N_1} \log(1 - P(y_j|\mathbf{x}_j))\right)$$

$$= \frac{1}{N}\sum_{i=1}^{N}\left(y_i \cdot \log P(y_i|\mathbf{x}_i) + (1 - y_i) \cdot \log(1 - P(y_i|\mathbf{x}_i))\right)$$

Logistic Regression with MLE

$* \; 1-\sigma(s) = 1 - \frac{1}{1+e^{-s}} = \frac{e^{-s}}{1+e^{-s}}$

- **Objective function for logistic regression (re-visit)**

$$L = \frac{1}{N}\sum_{i=1}^{N}[\,-y_i \cdot \log\sigma(s_i) - (1-y_i)\cdot\log\big(1-\sigma(s_i)\big)\,]$$

$$= \frac{1}{N}\sum_{i=1}^{N}[\,y_i \cdot \log(1+e^{-s_i}) + (y_i-1)\cdot\log\left(\frac{e^{-s_i}}{1+e^{-s_i}}\right)]$$

$$= \frac{1}{N}\sum_{i=1}^{N}[\,y_i \cdot \log(1+e^{-s_i}) + (y_i-1)\cdot(\log e^{-s_i} - \log(1+e^{-s_i}))\,]$$

$$= \frac{1}{N}\sum_{i=1}^{N}[\,y_i \cdot \log e^{-s_i} - \log e^{-s_i} + \log(1+e^{-s_i})\,]$$

$$= \frac{1}{N}\sum_{i=1}^{N}[-y_i \cdot s_i + \log(\,e^{s_i}+1)\,] = \frac{1}{N}\sum_{i=1}^{N}[-y_i \cdot \left(\mathbf{w}^{\mathbf{T}}\mathbf{x}+b\right) + \log\left(\,e^{\mathbf{w}^{\mathbf{T}}\mathbf{x}+b}+1\right)]$$

Logistic Regression with MLE

- **As a result, we have the same optimization problem with the objective function derived from the cross-entropy loss**

- **After then, the same story follows…**
  - Partial derivative computation
  - Gradient descent
  - …

Thank You